HIGH-LEVEL DESIGN DOCUMENT

Task Management Web Application

Waterfall Software Development Model

Document Version	v1.0
Project Code	TMA-2026
Prepared By	Architecture Team
Date	March 14, 2026
Status	APPROVED
Classification	Internal / Confidential

Document Revision History

Version	Date	Description	Author
1.0	14 Mar 2026	Initial HLD baseline	Architecture Team
0.2	10 Mar 2026	Added deployment & security sections	Lead Architect
0.1	03 Mar 2026	Draft for internal review	Architecture Team

Table of Contents

1. Introduction

1.1 Purpose

This High-Level Design (HLD) document defines the overall system architecture, component structure, technology stack, data model, API contracts, security posture, and deployment topology for the Task Management Web Application (TMA). It serves as the authoritative technical reference for the design phase of the Waterfall SDLC and provides the baseline from which the Low-Level Design (LLD) and implementation phases will proceed.

1.2 Project Overview

Project Name	Task Management Web Application (TMA)
Project Code	TMA-2026
Methodology	Waterfall (Sequential SDLC)
Project Phase	Phase 3 — Design (HLD)
Target Users	Internal team members of a small organisation (≤50 users)
Go-Live Target	Q3 2026
Project Sponsor	Chief Technology Officer

1.3 Scope

In Scope:

- User authentication and authorisation (login, registration, role management)
- Task creation, assignment, prioritisation, and status tracking
- Dashboard with summary metrics and activity feed
- Email & in-app notifications for task events
- REST API backend with JSON responses
- Responsive single-page front-end application
- Relational database persistence and Redis session caching

Out of Scope:

- Mobile native applications (iOS / Android)
- Third-party calendar integrations (future phase)
- AI-powered task suggestions
- Multi-tenancy / SaaS features

1.4 Definitions and Acronyms

Term / Acronym	Definition
HLD	High-Level Design — architectural design artefact in Waterfall SDLC
LLD	Low-Level Design — detailed component-level design artefact

SPA	Single-Page Application — front-end architecture served as static assets
API	Application Programming Interface
REST	Representational State Transfer — API architectural style
JWT	JSON Web Token — stateless authentication token standard (RFC 7519)
ORM	Object-Relational Mapper — library abstracting SQL queries
CI/CD	Continuous Integration / Continuous Deployment pipeline
VPC	Virtual Private Cloud — isolated network on AWS
WAF	Web Application Firewall

2. System Architecture Overview

2.1 Architectural Pattern

TMA adopts a three-tier client-server architecture separating presentation, application logic, and data concerns. Each tier is independently deployable, enabling targeted scaling and simplified maintenance.

2.2 Architecture Diagram

```
┌──────────────────────────────────────────────────────────┐
│                        CLIENT TIER                       │
│  [Browser / SPA]  ←──── HTTPS ────→  [Mobile Browser]    │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│                     PRESENTATION TIER                    │
│               React 18 SPA (Nginx static host)           │
└──────────────────────────────────────────────────────────┘
                              │ REST / JSON
                              ▼
┌──────────────────────────────────────────────────────────┐
│                      APPLICATION TIER                    │
│          Node.js / Express REST API  (Port 3000)         │
│  [Auth MW]  [Task Service]  [User Service]  [Notif SVC]  │
└──────────────────────────────────────────────────────────┘
        │ SQL                                   │ Cache
        ▼                                       ▼
┌──────────────────────┐                 ┌──────────────────────┐
│     DATA TIER        │                 │      CACHE TIER      │
│  PostgreSQL 16       │                 │  Redis (sessions)    │
└──────────────────────┘                 └──────────────────────┘
```

2.3 Tier Summary

Tier	Technology	Role
Client Tier	React 18, Tailwind CSS	Render UI, manage local state, call REST API over HTTPS
Presentation Tier	Nginx 1.25 (static host)	Serve compiled SPA bundles, perform TLS termination
Application Tier	Node.js 20 LTS, Express 4.x	Business logic, authentication, data validation, API routing
Data Tier	PostgreSQL 16	Persistent relational storage for all domain entities
Cache Tier	Redis 7 (ElastiCache)	JWT blocklist, session tokens, rate-limit counters

3. Component Design

3.1 Front-End Components

Component	Technology	Responsibility
Router	React Router v6	SPA route management; protected routes enforced via JWT presence check
Auth Module	Custom hooks + JWT	Login, registration, password reset; stores token in httpOnly cookie
Task Module	React + Zustand	CRUD forms, task list views, drag-and-drop board (Kanban)
Dashboard	Recharts	Summary KPI cards, activity timeline, task completion chart
Notification UI	React Toast lib	Real-time in-app alerts via WebSocket; dismissable toast queue
API Client	Axios + interceptor	Centralised HTTP client; attaches Bearer token, retries on 401

3.2 Back-End Services

Component	Technology	Responsibility
Auth Service	Express + bcrypt + JWT	Issues & validates JWT; manages refresh token rotation
Task Service	Express + Sequelize ORM	Full task lifecycle: create, assign, update, delete, filter, sort
User Service	Express + Sequelize ORM	User profiles, role assignment, team membership management
Notification SVC	Node-cron + Nodemailer	Sends email alerts on task events; queues in-app notifications
Middleware Stack	Helmet, CORS, Rate-limiter	Security headers, CORS policy, per-IP rate limiting (100 req/min)
Logger	Winston + Morgan	Structured JSON logs; request/response audit trail

4. Technology Stack

Layer	Technology	Version	Justification
Front-End Framework	React	18.3	Mature ecosystem, component reuse
Styling	Tailwind CSS	3.4	Utility-first, rapid UI iteration
State Management	Zustand	4.x	Lightweight, no boilerplate
Build Tool	Vite	5.x	Fast HMR, optimised bundles
API Server	Node.js	20 LTS	Non-blocking I/O, JS full-stack
Web Framework	Express	4.18	Minimal, well-understood
ORM	Sequelize	6.x	Migration support, PostgreSQL
Database	PostgreSQL	16	ACID compliance, rich SQL
Cache / Session	Redis	7	Sub-ms latency, pub/sub
Auth	JWT (jsonwebtoken)	9.x	Stateless, standard
Container	Docker	25	Reproducible environments
Orchestration	Docker Compose	2.x	Local dev & CI environment
CI/CD	GitHub Actions	Latest	Native SCM integration
Cloud	AWS (EC2, RDS, ElastiCache, CloudFront)	—	Cost-effective for small scale

5. Data Flow Design

5.1 Data Flow Diagram

The following diagram illustrates the primary data flows for user authentication and task retrieval.

```
User  ──[Login]──▶  Auth API  ──[Validate]──▶  DB
                      │                         │
                  [JWT Token]          [Session Store]
                      │                         │
User  ◀──────── [200 OK + Token] ◀──────────────┘

User  ──[GET /tasks]──▶  Task API  ──[SELECT]──▶  DB
                           │                       │
                       [Check Cache]         [Row data]
                           │                       │
User  ◀── [200 + JSON list] ◀── [Cache/Return] ◀──┘
```

5.2 Key Data Flows

Authentication Flow

- User submits credentials via the Login form (HTTPS POST /auth/login)
- Auth Service validates against hashed password in PostgreSQL
- On success: JWT access token (15 min) + refresh token (7 days) returned
- Refresh token stored in Redis; access token stored client-side in memory
- All subsequent API calls include Authorization: Bearer <token> header

Task CRUD Flow

- Authenticated user sends request to Task Service endpoint
- Middleware validates JWT signature and expiry
- Task Service executes Sequelize ORM query against PostgreSQL
- Response payload JSON-serialised and returned with HTTP 200/201/204
- Notification Service triggered asynchronously on task assignment/status change

6. Database Design

6.1 Entity-Relationship Summary

The TMA database is relational (PostgreSQL). The core entities are Users, Tasks, Projects, Comments, and Notifications. All tables include created_at and updated_at audit columns managed by Sequelize hooks.

6.2 Users Table

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique user identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt hashed password
full_name	VARCHAR(100)	NOT NULL	Display name
role	ENUM	DEFAULT 'member'	admin | manager | member
is_active	BOOLEAN	DEFAULT TRUE	Soft-delete flag
created_at	TIMESTAMPTZ	NOT NULL	Record creation timestamp
updated_at	TIMESTAMPTZ	NOT NULL	Last update timestamp

6.3 Projects Table

Column	Type	Constraints	Description
id	UUID	PK	Unique project identifier
name	VARCHAR(150)	NOT NULL	Project display name
description	TEXT	NULLABLE	Project description
owner_id	UUID	FK → users.id	Project owner
status	ENUM	DEFAULT 'active'	active | archived | completed
created_at	TIMESTAMPTZ	NOT NULL	Record creation timestamp

6.4 Tasks Table

Column	Type	Constraints	Description
id	UUID	PK	Unique task identifier

title	VARCHAR(200)	NOT NULL	Task title
description	TEXT	NULLABLE	Detailed task description
project_id	UUID	FK → projects.id	Parent project
assignee_id	UUID	FK → users.id	Assigned team member
reporter_id	UUID	FK → users.id	Task creator
status	ENUM	DEFAULT 'todo'	todo | in_progress | review | done
priority	ENUM	DEFAULT 'medium'	low | medium | high | critical
due_date	DATE	NULLABLE	Task deadline
created_at	TIMESTAMPTZ	NOT NULL	Record creation timestamp
updated_at	TIMESTAMPTZ	NOT NULL	Last update timestamp

7. API Design

7.1 API Conventions

- Base URL: https://api.tma.internal/v1
- All requests and responses use Content-Type: application/json
- Authentication: Bearer JWT in Authorization header (except /auth endpoints)
- Error responses follow RFC 7807 Problem Details format
- Pagination: ?page=1&limit=20 query parameters on list endpoints

7.2 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/auth/register	None	Register a new user account
POST	/auth/login	None	Authenticate and receive JWT tokens
POST	/auth/refresh	Refresh	Exchange refresh token for new access token
POST	/auth/logout	Bearer	Invalidate refresh token in Redis
POST	/auth/forgot-password	None	Trigger password reset email
PATCH	/auth/reset-password	Reset	Complete password reset with token

7.3 Task Endpoints

Method	Endpoint	Auth	Description
GET	/tasks	Bearer	List tasks with filters (project, status, priority, assignee)
POST	/tasks	Bearer	Create a new task
GET	/tasks/:id	Bearer	Retrieve a single task by ID
PUT	/tasks/:id	Bearer	Full update of a task record
PATCH	/tasks/:id	Bearer	Partial update (e.g., status change only)
DELETE	/tasks/:id	Bearer	Soft-delete a task
POST	/tasks/:id/comments	Bearer	Add a comment to a task

7.4 User & Project Endpoints

Method	Endpoint	Auth	Description
GET	/users	Bearer (admin)	List all users in the organisation
GET	/users/:id	Bearer	Retrieve user profile

PATCH	/users/:id	Bearer (self/admin)	Update user profile or role
GET	/projects	Bearer	List projects accessible to the caller
POST	/projects	Bearer (manager+)	Create a new project
PUT	/projects/:id	Bearer (owner)	Update project metadata
DELETE	/projects/:id	Bearer (admin)	Archive a project

8. Security Considerations

8.1 Authentication & Authorisation

- JWT access tokens expire after 15 minutes; refresh tokens after 7 days
- Refresh tokens stored in Redis with server-side invalidation on logout
- Role-Based Access Control (RBAC): admin > manager > member
- Password hashing via bcrypt with cost factor 12
- Account lockout after 5 consecutive failed login attempts (30-minute block)

8.2 Transport & API Security

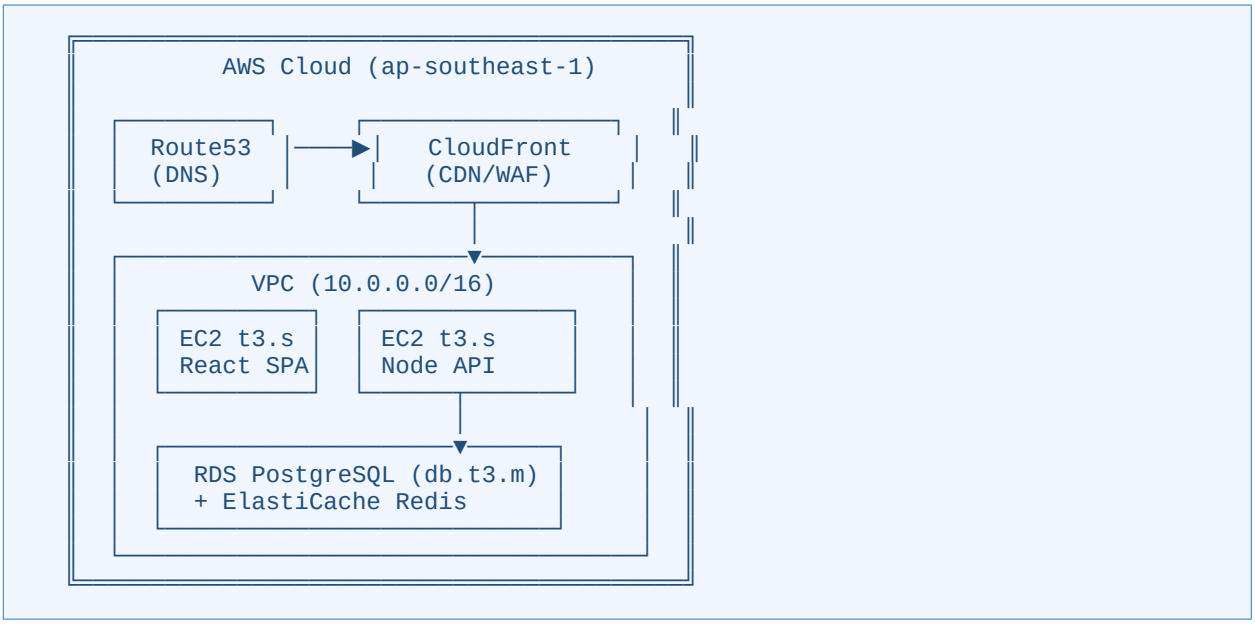
- HTTPS enforced via CloudFront + ACM certificate; HTTP redirected to HTTPS
- HSTS header with max-age=31536000; includeSubDomains
- Helmet.js sets X-Frame-Options, X-Content-Type-Options, CSP headers
- CORS restricted to listed origins only; credentials: true for cookie support
- Rate limiting: 100 requests/min per IP (Redis-backed); 10 req/min on /auth endpoints

8.3 Data Security

- All PII stored in PostgreSQL; database not publicly accessible (private subnet)
- Encryption at rest via AWS RDS storage encryption (AES-256)
- Environment secrets managed via AWS Secrets Manager; never committed to SCM
- SQL injection prevented by Sequelize parameterised queries
- Input validation via Joi schema on all POST/PUT/PATCH endpoints

9. Deployment Architecture

9.1 Deployment Diagram



9.2 Environment Strategy

Environment	Purpose	Infrastructure	Data
Development	Local dev & unit tests	Docker Compose (local)	Seed data only
Staging	Integration & UAT	AWS EC2 t3.small	Anonymised prod clone
Production	Live system (≤50 users)	AWS EC2 t3.small x2 (ALB)	Live data (encrypted)

9.3 CI/CD Pipeline

- Push to feature branch → GitHub Actions triggers lint + unit tests
- PR merge to main → build Docker image → push to Amazon ECR
- Deploy to staging automatically; production deploy requires manual approval gate
- Database migrations run via Sequelize CLI as a pre-deploy step
- Rollback: re-deploy previous ECR image tag via GitHub Actions workflow_dispatch

10. Risks & Assumptions

10.1 Technical Risks

Risk	Likelihood	Impact	Mitigation
PostgreSQL schema changes cause migration failures in prod	Low	High	Automated migration tests in staging; rollback scripts prepared
Redis unavailability causes session loss	Low	Medium	Graceful degradation to DB-based session fallback
JWT secret rotation disrupts active users	Low	Medium	Implement key ID (kid) claim for zero-downtime rotation
AWS EC2 instance single point of failure	Medium	High	ALB with two EC2 instances in separate AZs
Third-party npm package vulnerability	Medium	Medium	Dependabot alerts + weekly npm audit in CI
Over-permissive CORS settings expose API	Low	High	Strict origin whitelist enforced; reviewed per release

10.2 Assumptions

- Maximum concurrent users will not exceed 50 during initial rollout
- AWS ap-southeast-1 region is acceptable for data residency requirements
- SMTP relay service (e.g., AWS SES) is available for email notifications
- All team members have access to GitHub repository and AWS console
- Waterfall phase gates will be formally signed off before proceeding

11. Appendix

11.1 Related Documents

Document	Version	Status
Business Requirements Document (BRD)	1.0	Approved
Software Requirements Specification (SRS)	1.2	Approved
Low-Level Design (LLD)	TBD	Pending (next phase)
Test Plan	TBD	Pending
Deployment Runbook	TBD	Pending

11.2 Approval Sign-Off

Role	Name	Signature / Date
Lead Architect	_____	_____
Project Manager	_____	_____
CTO / Sponsor	_____	_____
QA Lead	_____	_____

— End of Document —